Meridian — Design Spec

Date: 2026-10-04 **Status:** awaiting user review **Path classification:** architectural

---

1. Purpose

Meridian is a Python-backed healthcare platform with three linked capabilities:

1. **Patient registry** — enter, retrieve and search patient records stored in CSV
2. **Symptom triage** — rank candidate diseases using set algebra over reported symptoms
3. **Cohort analytics** — profile real public clinical datasets with pandas and matplotlib

A separate frontend (built in a later phase) consumes a Flask JSON API.

Success criteria: every line of backend code traces to a concept from Software Lab-1 practicals 0–6, or is documented in `docs/concepts/` as an explicitly-justified extension.

---

2. Hard constraint: concept boundary

2.1 In scope — taught concepts and where they are used

Lab	Concept	Used for
P1	variables, 7 operator families, typecasting, `input()`	field validation, BMI/risk-band thresholds
P2	strings: indexing, slicing, `.split()`, `.strip()`, `.casefold()`, f-strings, `enumerate()`	CSV field parsing, query normalisation, report formatting
P2	lists: indexing, slicing, `+`, `*`, `.sort()`	visit history, ranked result lists
P2	tuples: indexing, packing/unpacking	multi-value returns from scoring functions
P3	sets: `&`, `|`, `-`, `^`, `.issubset()`	symptom matching, contraindication checks, store/medicine cross-lookup
P3	dicts: `get()`, `.keys()`, `.items()`, nested dicts	record storage, field weights, disease knowledge base
P3	`sorted(key=lambda)`, `max(key=lambda)`	ranking search and triage results
P4	functions: composition, `return`, docstrings, default args	every module's public API
P4	files: `open()` modes `r/w/a/x`, `with`, `readline`, `writelines`, `seek`	CSV persistence
P4	`os`: `listdir`, `walk`, `makedirs`, `path.exists`, `rename`	dataset discovery, timestamped backups
P4	classes: `__init__`, methods, `self`	`Patient`, `Visit`
P6	pandas: `read_csv`, `shape`, `describe`, `isnull`, `select_dtypes`, `groupby`, `value_counts`, `sort_values`, `iloc`, `loc`	cohort analytics

2.2 Out of scope — requires its own `.md` in `docs/concepts/`

Concept	Why needed	Doc file
Flask routing, `request.json`, `jsonify`	HTTP API layer	`docs/concepts/flask.md`
`csv.DictReader` / `DictWriter`	reading rows as dicts	`docs/concepts/csv-module.md`
`df.select_dtypes(include=...)`	separating numeric from categorical	`docs/concepts/pandas-select-dtypes.md`

`na_values` in `read_csv`	UCI `?` and `-9` markers	`docs/concepts/pandas-missing-values.md`
matplotlib `plt.hist`, `plt.bar`, `savefig`	charts	`docs/concepts/matplotlib-basics.md`
`requests.get`, `response.status_code`	dataset download	`docs/concepts/requests.md`
`json.dumps`, `json.loads`	writing `profile_report.csv` consumers may parse; reading Flask test fixtures	`docs/concepts/json.md`

2.3 Explicitly forbidden

- SQL or any database engine (CSV only, by design)
- Machine-learning model fitting (`sklearn`, `xgboost`) — the syllabus does not cover it
- Any frontend framework inside the Python process
- Third-party plotting beyond matplotlib

---

3. Architecture

```
D:/Q_project/projects/meridian\
├── backend/
│   ├── app.py          Flask routes only, no business logic
│   ├── store.py        CSV read/write, timestamped backup
│   ├── registry.py     partial-match search across 3 record types
│   ├── triage.py       set algebra, risk scoring, report rendering
│   ├── analytics.py    pandas profiling, matplotlib charts
│   └── models.py       Patient, Visit classes
├── data/
│   ├── patients.csv       one row per patient
│   ├── visits.csv         append-only, one row per consultation
│   ├── medicines.csv      curated catalogue
│   ├── stores.csv         medical stores/pharmacies
│   ├── diseases.csv       triage knowledge base
│   ├── datasets/          downloaded clinical CSVs + profile_report.csv
│   ├── backups/           timestamped registry snapshots
├── scripts/
│   ├── fetch_datasets.py  download candidates (requests)
│   ├── stage_datasets.py  attach headers, tidy column names (os)
│   └── profile_dataset.py profile + score against criteria (pandas)
├── docs/concepts/      one .md per out-of-syllabus concept
└── frontend/           phase 2
```

Layering rule: `app.py` contains routing and nothing else. Every route delegates to exactly one function in another module. This keeps Flask the only thing needing its own `.md`, and makes the rest independently testable without a server.

3.1 Data flow

```
POST /api/patients        → store.add_patient()      → append to patients.csv
GET  /api/patients/<id>   → store.get_patient()      → DictReader → filter by id
                                                  → join visits.csv rows
GET  /api/search?q=&type= → registry.search_records()→ score, sort, return ranked list
POST /api/triage          → triage.rank_diseases()  → set ops over diseases.csv
GET  /api/analytics       → analytics.list_datasets()→ read profile_report.csv
GET  /api/analytics/<n>   → analytics.profile()      → describe/groupby/value_counts
GET  /api/analytics/<n>/charts → analytics.save_charts() → PNG to static/charts/
POST /api/backup          → store.backup_all()       copy 5 registries to backups/<ts>/
```

---

4. Data layer

4.1 Store shape — two files, not one per patient

`patients.csv` holds one row per patient keyed by `patient_id`. `visits.csv` is append-only, one row per consultation, many rows per patient.

Rationale: append-only is exactly the Lab 4 `'a'` mode pattern, requiring no new concept. Retrieval is `DictReader` into a list of dicts then a list comprehension filter — the Practical 6 boolean-mask logic expressed in pure Python. One file per patient would make the most-used operation (search) an O(n) disk scan.

4.2 The blank-cell rule

Every field is optional. Two rules govern this:

1. **A blank cell means unknown.** Never `"N/A"`, never `"unknown"`. Blank means the question was not asked, which differs from the answer being negative.
2. **Blank fields never score.** In `match_score`, an empty value is skipped entirely — neither match nor mismatch. Scoring a blank as a mismatch would push otherwise good records down the ranking.

4.3 Record schemas

patients.csv — `patient_id, name, age, gender, phone, blood_group, area, city, registered_on, notes`

visits.csv — `visit_id, patient_id, visit_date, symptoms, diagnosis, severity, notes`

medicines.csv — `medicine_id, name, generic, category, form, strength, otc, rx_required, storage, price, manufacturer`

stores.csv — `store_id, name, type, area, city, phone, hours, rating, stock_csv`

diseases.csv — `disease_id, name, severity, symptoms, medication, dosage, age_range, contraindications`

4.4 Why symptoms are pipe-delimited

The `symptoms` cell holds values joined by `|`, not `,`:

```
fatigue|thirst|frequent urination|blurred vision|weight loss
```

The Practical 3 cricket-news problem already demonstrated the failure mode this avoids: searching for `"Sri"` and `"Lanka"` as separate tokens because country names contain spaces. Pipe-delimiting sidesteps that with a `split("|")` pattern already written in Lab 2.

`age_range` uses the same convention: `"35-70"`.

---

5. Triage scoring

5.1 Loading

```
def load_diseases(path="data/diseases.csv"):
    diseases = []
    with open(path, "r", newline="") as f:
        for row in csv.DictReader(f):
            row["symptom_set"] = set(row["symptoms"].split("|"))
            diseases.append(row)
    return diseases
```

5.2 Scoring — three numbers, not one ratio

```
def score_disease(patient_symptoms, disease):
    if not patient_symptoms:
```

```
        return 0.0, 0.0, 0.0, 0, len(disease["symptom_set"])

    matched   = patient_symptoms & disease["symptom_set"]
    extra     = patient_symptoms - disease["symptom_set"]
    missing   = disease["symptom_set"] - patient_symptoms

    coverage  = len(matched) / len(disease["symptom_set"])
    precision = len(matched) / len(patient_symptoms)
    penalty   = len(extra) * 0.5

    score = (coverage * 0.6) + (precision * 0.4) - penalty
    score = max(0.0, min(1.0, score))     # clamp: a long unrelated symptom list
                                           # must not produce a negative percentage
    return score, coverage, precision, len(matched), len(missing)
```

Every operation is `&` or `-` on two sets — the Lab 3 Venn practical, minus the films.

Why three numbers. A patient reporting 14 symptoms must not match Common Cold (5 symptoms) just because 4 of 5 matched; `precision` punishes that. Conversely a patient reporting only `fever|fatigue|cough` must not be told it is nothing, even though `coverage` alone would rank Dengue (7 symptoms) below Malaria (4). The weighted blend avoids both failure modes.

Why the clamp. Without it, a patient reporting six symptoms unrelated to any disease in the base accumulates `penalty` larger than the coverage contribution and scores below zero, which would render as a negative percentage. Clamping to `[0, 1]` keeps the report honest: no match is 0%, not −40%.

Empty input. An empty symptom list returns zeros rather than dividing by zero. The API layer rejects it earlier with HTTP 400; this guard exists so the function is safe when called directly.

5.3 Severity is a band, not a multiplier

Severity does not multiply the score. A `high` severity disease at score 0.55 outranks a `low` severity disease at 0.60. Multiplying would let a low-severity score climb arbitrarily high, which is clinically wrong — severity changes urgency, not likelihood.

```
SEVERITY_RANK = {"low": 0, "moderate": 1, "elevated": 2, "high": 3, "severe": 4}
BAND_WIDTH = 0.05

def apply_severity_bands(results):
    """Sort by score, then promote higher severity within a narrow band."""
    results.sort(key=lambda r: r["score"], reverse=True)

    banded = []
    i = 0
    while i < len(results):
        band = []
        anchor = results[i]["score"]
        while i < len(results) and anchor - results[i]["score"] < BAND_WIDTH:
            band.append(results[i])
            i += 1
        band.sort(key=lambda r: SEVERITY_RANK.get(r["severity"], 0),
                  reverse=True)
        banded.extend(band)
    return banded
```

The algorithm walks the score-sorted list, collects runs whose scores sit within `BAND_WIDTH` of the run's first element, then re-sorts each run by severity. So a disease at 0.58 outranks one at 0.55 unless they fall in the same band, in which case severity decides. Ties in both score and severity keep their original score order, because Python's `sort` is stable.

5.4 Ranking

```
def rank_diseases(symptoms, conditions=None, age=None):
    patient_symptoms = {s.strip().casefold() for s in symptoms}
```

```
    conditions = conditions or set()
    results = []

    for d in load_diseases():
        score, cov, prec, n_match, n_miss = score_disease(patient_symptoms, d)
        if n_match < 2:
            continue
        warnings = check_contraindications(d, conditions)
        results.append({...})

    results.sort(key=lambda r: r["score"], reverse=True)
    return results[:5]
```

- The `{s.strip().casefold() ...}` set comprehension is Lab 2 string methods
- `sort(key=lambda ..., reverse=True)` is the Practical 3 `max(staff, key=...)` pattern
- `n_match < 2` is the minimum-evidence gate — a single shared symptom is not a signal
- `check_contraindications` intersects `diseases.contraindications` against reported conditions, same operator in the opposite direction

5.5 Report output

Reuses the Lab 5 banner format from the salary-slip practical:

```
**************************************
 MERIDIAN CONSULTATION
**************************************
Patient       : P-0007  (age 44)
Reported      : fever, fatigue, dry cough, joint pain

MOST LIKELY
  1. Dengue Fever         72%   match 4/6   SEVERE
     WARNING: Metformin is contraindicated — patient reports "kidney disease"
  2. Malaria              61%   match 3/4   HIGH
  3. Influenza           38%   match 2/5   MODERATE

**************************************
NOT A DIAGNOSIS. TRIAGE GUIDANCE ONLY.
**************************************
```

The disclaimer is mandatory and appears in both the console report and the API response.

---

6. Registry search

6.1 One function, three record types

All three registries share a field-name → weight mapping, so `search_records` serves medicines, stores and patients identically.

```
FIELD_WEIGHTS = {"name": 5, "generic": 4, "category": 3, "city": 2, "area": 2}

def normalise(text):
    return str(text).strip().casefold()

def match_score(record, query_terms):
    score = 0
    matched_fields = []
    for field in record:
        value = record[field]
        if value == "" or field.endswith("_id"):
            continue                    # blank-cell rule
        value = normalise(value)
        for term in query_terms:
            if term in value:
```

```
                score += FIELD_WEIGHTS.get(field, 1)
                matched_fields.append(field)
                break
    return score, matched_fields

def search_records(records, query):
    terms = [normalise(t) for t in query.split() if t.strip()]
    results = []
    for rec in records:
        score, fields = match_score(rec, terms)
        if score > 0:
            results.append({**rec, "score": score, "matched": fields})
    results.sort(key=lambda r: r["score"], reverse=True)
    return results
```

`FIELD_WEIGHTS` makes searching "Metformin" rank the correct medicine above a store that merely mentions it in a low-weight field — the Practical 3 `max(key=...)` idea applied to field importance.

6.2 Weak matches are kept, not hidden

Records scoring above 0 are all returned, sorted. Nothing is silently dropped. The frontend greys out results below a visual threshold rather than the backend discarding them, because an empty result set reads as a bug when the pharmacy table simply holds three medicines and none is Metformin.

6.3 Cross-lookup: which stores stock a medicine

`stores.csv` stores its inventory as a `stock_csv` column of pipe-delimited medicine ids. `stores_with_medicine` splits that into a set per store, then intersects:

```
def stores_with_medicine(medicine_name, medicines, stores):
    target = normalise(medicine_name)

    med_ids = {m["medicine_id"] for m in medicines
               if target in normalise(m["name"])
               or target in normalise(m["generic"])}

    results = []
    for s in stores:
        if not s.get("stock_csv"):
            continue
        stock_set = set(s["stock_csv"].split("|"))
        if med_ids & stock_set:
            results.append(s)
    return results
```

One set comprehension, one intersection, one list comprehension. A store with a blank `stock_csv` is skipped rather than treated as stocking nothing — consistent with the blank-cell rule in section 4.2.

6.4 Search behaviour with no matches

An empty result list is a valid outcome, not an error. The response is HTTP 200 with `{"count": 0, "results": []}` so the frontend can render "no medicines matched 'xyz'" — distinct from a 400 for a blank query, and distinct from a 404 for a missing record id.

6.5 Entry and validation

`POST /api/patients`, `/api/medicines`, `/api/stores` each append one row. Validation is Lab 1 conditionals: `name` required; `age` and `price` must parse as numbers; blanks rejected with a reason string. Optional fields are written literally as empty strings.

---

7. Datasets

7.1 Selection procedure

Candidates were downloaded, staged with headers, and profiled by `scripts/profile_dataset.py`. Criteria, in priority order:

1. A usable target column (`target`, `Outcome`, `class`, `diagnosis`, `status`, `malignancy`)
2. At least 5 numeric columns
3. At least 150 rows
4. Under 20% missing cells

Thresholds were originally set at 200 rows and 10% missing. Both were loosened after the first profile run: Parkinsons failed on 195 vs 200 rows, which is not a quality signal, and 20% is defensible when the missingness is genuinely "not measured" rather than corruption.

7.2 Results

Dataset	Rows × Cols	Numeric	Missing	Target	Verdict
`pima_diabetes.csv`	768 × 9	8	0.00%	`Outcome` (0/1)	**Primary**
`heart_disease.csv`	303 × 14	13	0.14%	`target` (0–4)	**Secondary**
`parkinsons.csv`	195 × 24	22	0.00%	`status` (0/1)	**Tertiary**
`mammographic_masses.csv`	961 × 7	6	16.69%	`malignancy`	Excluded

Pima is primary — 768 rows, zero missing, binary target (500/268), and it links to Type 2 Diabetes in the triage knowledge base.

Heart is secondary — its 5-class ordinal target lets `groupby(target).mean()` do real work; `ca` separates classes at 1.32 spread, `oldpeak` at 1.15. A binary target cannot demonstrate that.

Parkinsons is tertiary — 22 numeric columns, `nhr` separates at 0.87. Proves the analytics generalise beyond one disease.

Masses is excluded — 16.69% missing fails the threshold. Retained on disk and documented as the case that motivates the blank-cell rule, since `?` there genuinely means "not measured".

7.3 Two bugs found by the profiler

Both are worth stating in a viva:

1. **Parkinsons double header.** The UCI archive ships a header row; the staging script attached a second. Detected because numeric columns read as 0 and the `status` column contained the literal string `"status"`.
2. **`?` and `-9` not parsed as NaN.** Without `na_values`, pandas read whole columns as object dtype and every numeric count was zero. After the fix, heart's missing rate is a truthful 0.14% rather than a false 0%.

7.4 Analytics endpoints

```
GET /api/analytics              → dataset list + profile_report.csv contents
GET /api/analytics/<name>       → describe(), value_counts(), groupby(target).mean()
GET /api/analytics/<name>/charts → matplotlib PNGs to static/charts/
```

7.5 Risk bands

Modelled on Practical 1 Batch 1 question B (BMI categorisation), same shape:

```
def risk_band(value, thresholds):
    if value < thresholds[0]: return "low"
    if value < thresholds[1]: return "moderate"
```

```
    if value < thresholds[2]: return "elevated"
    return "high"
```

Charts are explicit `plt.hist(...)` / `plt.bar(...)` calls rather than `df.hist()`, so every parameter (`bins=30`, `alpha=0.6`) can be explained.

---

8. Backup

`POST /api/backup` copies the five registry CSVs into `data/backups/<YYYYMMDD_HHMMSS_microseconds>/` using `shutil.copyfile`, and returns the paths written.

Scope is deliberately limited to the registries. An earlier version used `os.walk` over all of `data/`, which pulled in `data/datasets/` as well. Two files there share a name with registry files, so flattening everything into one destination folder silently overwrote one with the other. Scoping to a fixed list also keeps immutable research data out of routine backups. The microsecond timestamp suffix prevents two backups in the same second from colliding.

---

9. Error handling

Case	Behaviour
Missing CSV on first run	auto-create with header row, log the path
Malformed CSV row (wrong field count)	skip the row; list it in the response as `skipped: [{"line": 42, "reason": "expected 10 fields, got 9"}]`
Patient id not found	HTTP 404 with a clear message
Empty search query	HTTP 400 explaining the query was blank
Search with zero matches	HTTP 200 with `{"count": 0, "results": []}` — not an error
Empty symptom list in triage	HTTP 400 explaining at least one symptom is required
Dataset name not in profile report	HTTP 404 listing valid names
Contraindication hit	not an error — a `warnings` entry in a 200 response
Matplotlib chart failure	HTTP 500 with the exception message; PNG not written

The `skipped` list exists so that silently dropping a malformed row never hides data loss. If a CSV is written by this application it will always be well-formed, but the registries are plain files a user may edit by hand in a spreadsheet, so the read path must assume the possibility.

---

10. Testing

Level	Method	Covers
Unit — pure functions	direct calls, assert on return values	`normalise`, `score_disease`, `risk_band`, `match_score`
Unit — file I/O	write to `tmp/`, read back, assert equality	`add_patient`, `get_patient`, `backup_all`
Integration	`app.test_client()`	every route, both success and error paths
Manual	Flask dev server + browser	full demo path

No pytest — not covered by any lab, and adding it would need its own `.md`. Tests are plain functions with `assert`, run by a single script.

11. Explicit non-goals

- No authentication or user accounts
- No real patient data, ever — the registry is seeded with synthetic records
- No drug-interaction checking beyond single-drug contraindication lookup
- No mobile app; the frontend is responsive web only
- No deployment; this runs locally

---

12. Open items for the user

1. **Frontend stack** — deferred to phase 2. User indicated they will supply UI skills and animation guidance before that phase.
2. **Knowledge base size** — spec assumes ~15 curated diseases. Confirm this is enough for a convincing demo, or raise it.
3. **`diseases.csv` sourcing** — every row must be verifiable by the user. If any row cannot be sourced, it gets removed rather than invented.